Sesión 5
MODELOS CLÁSICOS PARA NLP

Naive Bayes, SVM, HMM, MaxEnt y CRF

Por
Carlos César Sánchez Coronel

2026

Índice general

Índice general **2**

1. Modelos Clásicos para NLP **3**
1.1. Logro de la sesión . . . 3
1.2. Introducción: la relevancia perdurable de los modelos clásicos . . . 3
1.3. Naive Bayes para Clasificación de Texto . . . 4
1.3.1. Fundamento matemático . . . 4
1.3.2. Variantes para texto . . . 4
1.3.3. Estimación de parámetros . . . 4
1.3.4. Ejemplo numérico . . . 4
1.3.5. Ventajas y limitaciones . . . 5
1.4. Support Vector Machines (SVM) para Texto . . . 5
1.4.1. Fundamento matemático (recapitulación) . . . 5
1.4.2. Por qué SVM funciona bien en texto . . . 5
1.4.3. Pipeline típico . . . 5
1.4.4. Ejemplo con código . . . 5
1.4.5. Ventajas y limitaciones . . . 6
1.5. Modelos Ocultos de Markov (HMM) para Datos Secuenciales . . . 6
1.5.1. Definición formal . . . 6
1.5.2. La propiedad de Markov . . . 6
1.5.3. Tres problemas fundamentales y sus algoritmos . . . 7
1.5.4. Aplicación en NLP: POS tagging . . . 7
1.5.5. Aplicaciones adicionales de HMM en NLP . . . 7
1.5.6. Ventajas y limitaciones . . . 8
1.6. Modelos de Máxima Entropía (MaxEnt / Regresión Logística Multinomial) 8
1.6.1. Fundamento matemático . . . 8
1.6.2. Características (features) . . . 8
1.6.3. Entrenamiento . . . 8
1.6.4. Aplicación en NLP . . . 8
1.6.5. Ventajas y limitaciones . . . 9
1.7. Conditional Random Fields (CRF) . . . 9
1.7.1. De HMM a CRF . . . 9
1.7.2. Definición formal . . . 9
1.7.3. Tipos de características . . . 9
1.7.4. Ventajas sobre HMM . . . 9
1.7.5. Implementación con sklearn-crfsuite . . . 10
1.7.6. Aplicaciones en NLP . . . 10
1.8. Comparación de Modelos Clásicos para NLP . . . 10

1.9. Caso Integrado: Clasificación de Secuencias y Etiquetado 11
1.9.1. Problema 11
1.9.2. Solución con modelos clásicos 11
1.9.3. Resultados esperados 11
1.10. Preparación para Entrevistas 12
1.10.1. Preguntas típicas 12
1.11. Conexión con el resto del curso 12
1.12. Resumen 12

Capítulo 1

Modelos Clásicos para NLP

En las sesiones anteriores se han cubierto las representaciones del texto (BOW, TF-IDF, embeddings). Ahora se aborda cómo utilizar esas representaciones para resolver tareas de NLP mediante algoritmos de machine learning clásicos. Estos modelos, aunque han sido superados en precisión por el deep learning, siguen siendo fundamentales por su interpretabilidad, eficiencia y buen rendimiento en datos pequeños. Esta sesión cubre Naive Bayes, SVM, Modelos Ocultos de Markov (HMM), Máxima Entropía (MaxEnt) y Conditional Random Fields (CRF), con énfasis en sus fundamentos matemáticos y aplicaciones en texto.

1.1. Logro de la sesión

Aplicar algoritmos de machine learning clásicos a tareas de NLP como clasificación de texto y etiquetado de secuencias, comprendiendo sus fundamentos matemáticos, sus ventajas y limitaciones frente a enfoques de deep learning.

1.2. Introducción: la relevancia perdurable de los modelos clásicos

Aunque el deep learning domina la investigación actual en NLP, los modelos clásicos siguen siendo extremadamente relevantes por varias razones:

- **Eficiencia con datos pequeños**: funcionan bien cuando no se dispone de millones de ejemplos.
- **Interpretabilidad**: sus decisiones son más fáciles de explicar y depurar.
- **Línea base obligatoria**: todo proyecto serio debe comparar contra estos modelos.
- **Producción en entornos restringidos**: en sistemas embebidos o de baja latencia, su simplicidad computacional es una ventaja.

1.3. Naive Bayes para Clasificación de Texto

1.3.1. Fundamento matemático

Naive Bayes aplica el teorema de Bayes con un supuesto de independencia condicional entre las características dada la clase. Para un documento d representado por sus palabras $w_1, w_2, ..., w_n$:

$$P(c|d) = \frac{P(c) \prod_{i=1}^{n} P(w_i|c)}{P(d)}$$

1.3.2. Variantes para texto

- **MultinomialNB**: asume que las palabras siguen una distribución multinomial (frecuencias). Es la más común para texto.
- **BernoulliNB**: asume variables binarias (presencia/ausencia).
- **GaussianNB**: para características continuas (no típico en texto crudo).

1.3.3. Estimación de parámetros

La probabilidad de una palabra w dada una clase c se estima con suavizado Laplace para evitar probabilidades cero:

$$P(w|c) = \frac{\text{freq}(w, c) + \alpha}{\sum_{w' \in V} \text{freq}(w', c) + \alpha|V|}$$

donde α es el parámetro de suavizado (típicamente 1).

1.3.4. Ejemplo numérico

Corpus de 3 documentos:

$$\begin{aligned} d_1 &: \text{"el gato caza ratones"} \quad \text{(clase: animales)} \\ d_2 &: \text{"el perro persigue gatos"} \quad \text{(clase: animales)} \\ d_3 &: \text{"el coche acelera rápido"} \quad \text{(clase: vehículos)} \end{aligned}$$

Vocabulario: el, gato, caza, ratones, perro, persigue, gatos, coche, acelera, rápido
Queremos clasificar "gato rápido". Calculamos:
Para clase *animales*:

$$\begin{aligned} P(\text{animales}) &= 2/3 \\ P(\text{gato}|\text{animales}) &= (1+1)/(6+10) = 2/16 = 0.125 \\ P(\text{rápido}|\text{animales}) &= (0+1)/(6+10) = 1/16 = 0.0625 \\ P(\text{animales}|d) &\propto 0.667 \times 0.125 \times 0.0625 = 0.00521 \end{aligned}$$

Para clase *vehículos*:

$$\begin{aligned} P(\text{vehículos}) &= 1/3 \\ P(\text{gato}|\text{vehículos}) &= (0+1)/(4+10) = 1/14 \approx 0.0714 \\ P(\text{rápido}|\text{vehículos}) &= (1+1)/(4+10) = 2/14 \approx 0.1429 \\ P(\text{vehículos}|d) &\propto 0.333 \times 0.0714 \times 0.1429 = 0.00340 \end{aligned}$$

El documento se clasifica como "animales" (probabilidad mayor).

1.3.5. Ventajas y limitaciones

- **Ventajas**: extremadamente rápido, funciona bien con datos pequeños, maneja alta dimensionalidad.
- **Limitaciones**: asume independencia (violado en lenguaje), no captura relaciones entre palabras.

1.4. Support Vector Machines (SVM) para Texto

1.4.1. Fundamento matemático (recapitulación)

SVM busca el hiperplano que maximiza el margen entre clases. En su forma lineal:

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n} \xi_i$$

sujeto a $y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1 - \xi_i$, $\xi_i \geq 0$.

1.4.2. Por qué SVM funciona bien en texto

- **Alta dimensionalidad**: los textos tienen miles de características, y SVM es efectivo cuando $p \gg n$.
- **Representaciones dispersas**: SVM maneja bien la esparsidad de BOW/TF-IDF.
- **Kernel lineal suficiente**: para texto, el kernel lineal suele ser suficiente, y es más rápido e interpretable.

1.4.3. Pipeline típico

1. Representar documentos con TF-IDF.
2. Normalizar vectores (L2).
3. Entrenar SVM lineal (ej. `LinearSVC` en scikit-learn).

1.4.4. Ejemplo con código

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.svm import LinearSVC
from sklearn.pipeline import make_pipeline

# Datos
docs = ["el gato caza ratones", "el perro persigue gatos", "el
   coche acelera r pido"]
y = ["animales", "animales", "veh culos"]
```

```
# Pipeline
model = make_pipeline(
    TfidfVectorizer(),
    LinearSVC(C=1.0)
)
model.fit(docs, y)
```

1.4.5. Ventajas y limitaciones

- **Ventajas**: robusto, buen rendimiento, maneja bien el desbalance (con pesos).
- **Limitaciones**: menos interpretable que Naive Bayes, sensible a la escala (requiere normalización).

1.5. Modelos Ocultos de Markov (HMM) para Datos Secuenciales

Los HMM son especialmente útiles cuando la tarea involucra secuencias (POS tagging, NER, segmentación).

1.5.1. Definición formal

Un HMM es un modelo probabilístico que describe una secuencia de observaciones asumiendo que existe un proceso oculto (estados) que sigue la propiedad de Markov. Se define por:

- **Estados ocultos** $Q = \{q_1, q_2, ..., q_N\}$: variables no observables (ej. etiquetas gramaticales).
- **Observaciones** $O = \{o_1, o_2, ..., o_T\}$: datos visibles (ej. palabras).
- **Matriz de transición** $A = [a_{ij}]$: probabilidad de pasar del estado i al estado j: $a_{ij} = P(q_{t+1} = j | q_t = i)$.
- **Matriz de emisión** $B = [b_j(k)]$: probabilidad de observar o_k en estado j: $b_j(k) = P(o_t = k | q_t = j)$.
- **Distribución inicial** $\pi = [\pi_i]$: probabilidad de comenzar en estado i: $\pi_i = P(q_1 = i)$.

El modelo completo se denota como $\lambda = (A, B, \pi)$.

1.5.2. La propiedad de Markov

La propiedad fundamental establece que el estado futuro depende solo del estado actual, no de toda la historia:

$$P(q_{t+1} | q_t, q_{t-1}, ..., q_1) = P(q_{t+1} | q_t)$$

Problema	Descripción	Algoritmo
Evaluación	Dado λ y una secuencia O, calcular $P(O\|\lambda)$	Forward-Backward
Decodificación	Dado λ y O, encontrar la secuencia de estados más probable	Viterbi
Aprendizaje	Dado O, estimar los parámetros λ	Baum-Welch (EM)

Cuadro 1.1: Tres problemas fundamentales en HMM.

1.5.3. Tres problemas fundamentales y sus algoritmos

1.5.4. Aplicación en NLP: POS tagging

En etiquetado gramatical:

- **Estados ocultos**: las etiquetas POS (NN, VB, JJ, etc.).
- **Observaciones**: las palabras.
- **Probabilidades de transición**: probabilidad de que un adjetivo (JJ) sea seguido por un sustantivo (NN).
- **Probabilidades de emisión**: probabilidad de que el estado "NN" emita la palabra "casa".

Entrenamiento supervisado con NLTK:

```
import nltk
from nltk.tag import HiddenMarkovModelTrainer
from nltk.corpus import treebank

# Datos de entrenamiento (oraciones etiquetadas)
train_data = treebank.tagged_sents()[:3900]

# Entrenar HMM
tagger = HiddenMarkovModelTrainer().train_supervised(train_data)

# Evaluar
accuracy = tagger.evaluate(treebank.tagged_sents()[3900:])
print(f"Precisi n: {accuracy:.2%}")  # ~96-98%
```

Decodificación con Viterbi:

```
sentence = "The cat sits on the mat".split()
tags = tagger.tag(sentence)
print(tags)
```

1.5.5. Aplicaciones adicionales de HMM en NLP

- **Reconocimiento de entidades nombradas**: estados ocultos = tipos de entidad (PERSON, LOC, ORG).
- **Segmentación de palabras**: para idiomas sin espacios explícitos.
- **Modelado de temas secuenciales**: en documentos largos.

- **Reconocimiento de voz**: estados = fonemas, observaciones = características acústicas.

1.5.6. Ventajas y limitaciones

- **Ventajas**: modelo generativo con fundamentos probabilísticos sólidos, interpretable, funciona bien con datos secuenciales etiquetados.
- **Limitaciones**: asume independencia de observaciones (dado el estado), limitado a dependencias de Markov de primer orden, requiere datos secuenciales etiquetados.

1.6. Modelos de Máxima Entropía (MaxEnt / Regresión Logística Multinomial)

1.6.1. Fundamento matemático

MaxEnt, equivalente a regresión logística multiclase, modela directamente la probabilidad condicional $P(c|d)$ sin asumir independencia:

$$P(c|d) = \frac{\exp\left(\sum_j \lambda_j f_j(c,d)\right)}{\sum_{c'} \exp\left(\sum_j \lambda_j f_j(c',d)\right)}$$

donde f_j son funciones características arbitrarias (pueden combinar palabras, contexto, etc.), y λ_j son los pesos aprendidos.

1.6.2. Características (features)

A diferencia de Naive Bayes, MaxEnt permite:

- **Características arbitrarias**: no necesitan ser independientes.
- **Características de contexto**: por ejemplo, "la palabra anterior es 'el'".
- **Características complejas**: combinaciones de palabras.

1.6.3. Entrenamiento

Se minimiza la log-verosimilitud negativa con regularización:

$$\mathcal{L} = -\sum_i \log P(c_i|d_i) + \frac{1}{2C}\sum_j \lambda_j^2$$

1.6.4. Aplicación en NLP

- **Clasificación de texto**: como alternativa a Naive Bayes, a menudo con mejor rendimiento.
- **POS tagging**: como modelo local (no secuencial).

1.6.5. Ventajas y limitaciones

- **Ventajas**: modelo discriminativo, no asume independencia, flexible en definición de características.
- **Limitaciones**: no modela secuencias explícitamente (ignora dependencias entre etiquetas consecutivas).

1.7. Conditional Random Fields (CRF)

1.7.1. De HMM a CRF

Mientras que HMM es un modelo **generativo** (modela $P(X, Y)$), CRF es un modelo **discriminativo** que modela directamente $P(Y|X)$ para secuencias. Esto permite relajar el supuesto de independencia de las observaciones.

1.7.2. Definición formal

Para una secuencia de etiquetas $Y = (y_1, ..., y_T)$ y observaciones $X = (x_1, ..., x_T)$:

$$P(Y|X) = \frac{1}{Z(X)} \exp\left(\sum_{t=1}^{T} \sum_{k} \lambda_k f_k(y_t, y_{t-1}, X, t)\right)$$

donde:

- f_k son funciones característica (pueden depender de toda la secuencia X).
- λ_k son pesos aprendidos.
- $Z(X)$ es la función de partición (normalización).

1.7.3. Tipos de características

- **Características de estado**: relacionan la etiqueta actual con las observaciones (ej. "si la palabra actual termina en -ción, es probable NN").
- **Características de transición**: relacionan la etiqueta actual con la anterior (ej. "es poco probable que DT siga a JJ").

1.7.4. Ventajas sobre HMM

- No asume independencia de las observaciones.
- Puede incluir características arbitrarias y superpuestas.
- Mejor rendimiento en tareas como NER y chunking.

1.7.5. Implementación con sklearn-crfsuite

```
import sklearn_crfsuite
from sklearn_crfsuite import metrics

# Caracter sticas: palabra actual, anterior, siguiente, sufijos,
    etc.
def word2features(sent, i):
    word = sent[i][0]
    features = {
        'word': word,
        'is_capitalized': word[0].isupper(),
        'suffix-2': word[-2:],
        'suffix-3': word[-3:],
        'prev_word': '' if i == 0 else sent[i-1][0],
        'next_word': '' if i == len(sent)-1 else sent[i+1][0],
    }
    return features

# Entrenar
crf = sklearn_crfsuite.CRF(
    algorithm='lbfgs',
    c1=0.1,  # L1 regularization
    c2=0.1,  # L2 regularization
    max_iterations=100
)
crf.fit(X_train, y_train)

# Evaluar
y_pred = crf.predict(X_test)
print(metrics.flat_f1_score(y_test, y_pred, average='weighted'))
```

1.7.6. Aplicaciones en NLP

- **Reconocimiento de Entidades Nombradas (NER)**.
- **Chunking (sintagmas)**.
- **Extracción de información**.
- **POS tagging** (aunque HMM suele ser suficiente).

1.8. Comparación de Modelos Clásicos para NLP

¿Cuándo usar cada uno?

- **Clasificación simple (no secuencial)**: Naive Bayes (baseline rápido), SVM (mejor rendimiento).
- **Etiquetado secuencial**: HMM si hay pocos datos y se necesita interpretabilidad; CRF si se dispone de muchas características y datos.

Modelo	Tipo	Datos	Velocidad	Interpretabilida
Naive Bayes	Generativo	Pequeños	Muy rápida	Alta
SVM	Discriminativo	Medianos	Rápida	Media
HMM	Generativo secuencial	Secuenciales etiquetados	Media	Alta
MaxEnt	Discriminativo	Medianos	Media	Media
CRF	Discriminativo secuencial	Secuenciales	Lenta	Baja

Cuadro 1.2: Comparación de modelos clásicos para NLP.

- **Cuando la independencia de palabras es insostenible**: MaxEnt o CRF.

1.9. Caso Integrado: Clasificación de Secuencias y Etiquetado

1.9.1. Problema

Tenemos un corpus de reseñas de productos, cada una con:

- Texto de la reseña.
- Calificación (1-5 estrellas) → clasificación de sentimiento.
- Además, queremos extraer automáticamente los aspectos mencionados (ej. "batería", "pantalla", "precio") → tarea de NER.

1.9.2. Solución con modelos clásicos

1. **Clasificación de sentimiento**: SVM con TF-IDF (baseline) y comparar con MaxEnt.
2. **Extracción de aspectos (NER)**:
   - Entrenar un HMM con datos etiquetados a nivel de palabra (aspecto vs no aspecto).
   - Comparar con un CRF que use características más ricas (contexto, forma de la palabra, etc.).
3. **Evaluación**:
   - Sentimiento: precisión, F1.
   - NER: F1 por entidad, exactitud de secuencia.

1.9.3. Resultados esperados

- SVM superará a Naive Bayes en sentimiento.
- CRF superará a HMM en NER (por poder usar características arbitrarias), pero requerirá más datos y tiempo de entrenamiento.

1.10. Preparación para Entrevistas

1.10.1. Preguntas típicas

- **"Menciona dos algoritmos clásicos para clasificación de textos y compáralos."** Naive Bayes (rápido, asume independencia) vs SVM (robusto, no asume independencia, mejor rendimiento).
- **"¿Cuándo usarías HMM en lugar de CRF?"** HMM: cuando hay pocos datos, se necesita interpretabilidad, o se quiere un modelo generativo. CRF: cuando se pueden definir características arbitrarias y se dispone de suficientes datos.
- **"Explica el algoritmo de Viterbi."** Algoritmo de programación dinámica que encuentra la secuencia de estados más probable en un HMM, basado en maximizar $P(\text{estados}|\text{observaciones})$.
- **"¿Qué es el suavizado Laplace y por qué se usa?"** Técnica para evitar probabilidades cero en Naive Bayes, añadiendo una pequeña constante a todas las frecuencias.
- **"¿Por qué SVM con kernel lineal funciona bien en texto?"** Porque los vectores de texto son de alta dimensionalidad y esparsos, lo que hace que los datos sean aproximadamente separables linealmente.
- **"¿Qué son las funciones característica en CRF?"** Funciones que pueden depender de toda la secuencia de entrada y de etiquetas vecinas, permitiendo modelar dependencias complejas.

1.11. Conexión con el resto del curso

- **Sesión 4 (Word Embeddings)**: los embeddings pueden alimentar a estos modelos como características densas.
- **Sesión 6 (RNN/LSTM)**: los modelos secuenciales neuronales superan a HMM/CRF en muchos problemas, pero requieren más datos.
- **Sesión 7 (Atención/Transformers)**: la evolución natural de los modelos secuenciales.

1.12. Resumen

- **Naive Bayes** y **SVM** son los caballos de batalla para clasificación de texto, con diferentes compensaciones.
- **HMM** modela secuencias probabilísticamente, con aplicaciones en POS tagging, NER y reconocimiento de voz.
- **MaxEnt** (regresión logística) permite características arbitrarias sin independencia.
- **CRF** extiende MaxEnt a secuencias, siendo el estado del arte en muchas tareas de etiquetado antes del deep learning.

- La elección del modelo depende del tipo de tarea, la cantidad de datos y la necesidad de interpretabilidad.